

CONTENIDO

Pruebas de correctitud de un algoritmo [G1.6,G2.3]. Funciones: propiedades [H2.3], Cardinalidad de conjuntos. Conjuntos contables e incontables [H2.4]. Diagonalización [H2.4]. Límites de la computabilidad [H2.4.4]. Material suplementario: funciones de cifrados, funciones hash [H2.3], funciones recursivas [H3.2].

GERSTING, JUDITH L. “Mathematical Structures for Computer Science: A Modern Approach to Discrete Mathematics”. W H Freeman & Co, 2014.

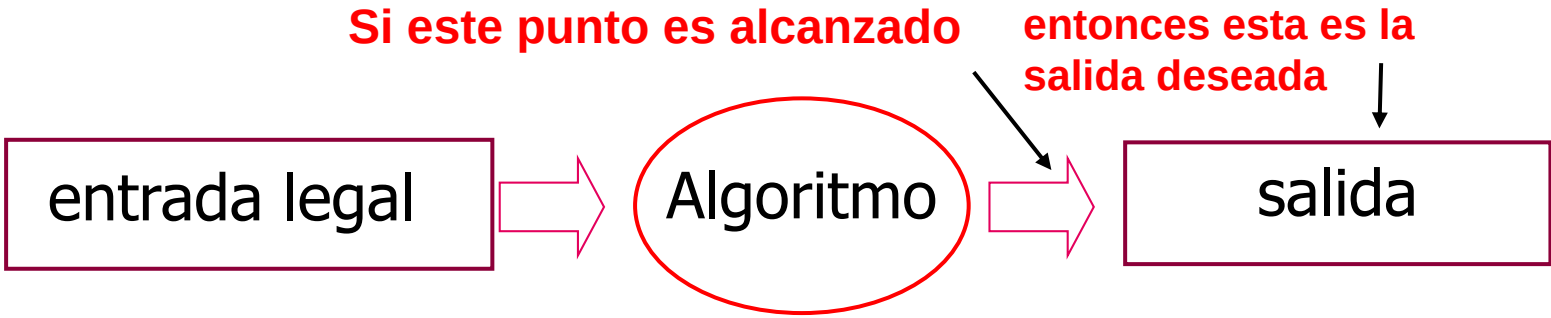
HEIN, JAMES. Discrete Structures, Logic and Computability. Jones and Bartlett Publishers. 2015

Un *algoritmo es correcto* si para cualquier entrada (input) legal termina y produce la salida (output) deseada.

Las pruebas de correctitud no pueden ser automatizadas.

Pero existen técnicas prácticas y formalismos rigurosos que ayudan a razonar sobre la correctitud de los algoritmos.

Correctitud parcial



Correctitud total



Para probar correctitud parcial asociamos una serie de aserciones (sentencias sobre el estado de la ejecución) a puntos específicos del algoritmo.

Precondiciones: aserciones que deben ser válidas antes de ejecutarse un algoritmo o subrutina.

Poscondiciones: aserciones que deben ser válidas luego de ejecutarse un algoritmo o subrutina.

$\{Q\}P\{R\}$ es llamada tripla de Hoare.

Q: precondition para el programa P

R: poscondition para el programa P

$\{Q\}$ P es el programa
 S_0 compuesto por
 $\{R_1\}$ las sentencias
 $S_1, S_1, \dots, S_{n-1}$
 S_1^1
 $\{R_2\}$
 $\vdots$
 S_{n-1}
 $\{R\}$

Para probar la correctitud de P
se debe probar que para toda
entrada X:

$Q(X)$ implica $R(X, P(X))$



Anthony Hoare
(nacido en 1934)

$$\begin{array}{c} \{Q\} \\ S_0 \\ \{R_1\} \\ S_1 \\ \{R_2\} \\ \vdots \\ S_{n-1} \\ \{R\} \end{array}$$

P es el programa
compuesto por
las sentencias
 $S_0, S_1, \dots, S_{n-1}$

Para probar
que P
es correcto

$$\left\{ \begin{array}{c} \{Q\} \quad S_0 \quad \{R_1\} \\ \{R_1\} \quad S_1 \quad \{R_2\} \\ \{R_2\} \quad S_2 \quad \{R_3\} \\ \vdots \\ \{R_{n-1}\} \quad S_{n-1} \quad \{R\} \end{array} \right.$$

$$\longrightarrow \{R_i\} x=e \{R_{i+1}\}$$

Ejemplo

$$\{x-1>0\}$$

$$x=x-1$$

$$\{x>0\}$$

$$\{x-1 > 0\} x =x-1 \{x>0\}$$

if condición B then

P1

else

P2

end if

$$\begin{array}{l} \{Q \wedge B\} \quad P_1 \quad \{R\} \\ \{Q \wedge \neg B\} \quad P_2 \quad \{R\} \end{array} \quad \longrightarrow \quad \{Q\} \quad S_i \quad \{R\}$$

Ejemplo

{n=5}

if n >= 10 then

y = 100

else

y = n + 1

end if

{y=6}

{n = 5 ∧ n >= 10} y=100 {y=6}

{n = 5 ∧ n < 10} y=n+1 {y=6}

{n = 5} y=n+1 {y=6}

while condición B do

P

end while

$$\{Q \wedge B\} P \{Q\} \longrightarrow \{Q\} s_i \{Q \wedge \neg B\}$$

Q se llama invariante
de ciclo

Ejemplo

Producto(enteros no negativos x e y)

variables locales: enteros i,j

i=0

j=0

while i<> x do

j=j+y

i=i+1

end while

return j

Q: j=i*y

Prueba de $Q: j=i*y$ utilizando inducción

Caso base:

$$Q(0): j_0=i_0*y$$

Hipótesis inductiva:

$$\text{Asumamos } Q(k): j_k=i_k*y$$

Paso inductivo:

$$\text{Probemos } Q(k+1): j_{k+1}=i_{k+1}*y$$

$$\text{Por el programa } j_{k+1}=j_k+y, \quad i_{k+1}=i_k+1$$

$$\begin{aligned} \text{Luego } j_{k+1} &= j_k+y && \text{(por programa)} \\ &= i_k*y+y && \text{(por HI)} \\ &= (i_k+1)*y \\ &= i_{k+1}*y && \text{(por programa)} \end{aligned}$$

Producto(enteros no negativos x e y)

variables locales: enteros i, j

$i=0$

$j=0$

while $i \leq x$ do

$j=j+y$

$i=i+1$

end while

return j

Probamos que la invariante es

$Q: j=i*y.$

Al finalizar el ciclo while se verifica

$i=x.$

Por lo tanto la función computa

$j = x * y.$

¿Cómo encontrar el máximo común divisor de dos enteros no negativos a y b , donde a y b no son simultáneamente 0?



Euclides
(300 AC)

El algoritmo de Euclides se basa en el siguiente hecho:

$$(\forall a, b, q, r) [(a=qb+r) \rightarrow (\text{mcd}(a,b)=\text{mcd}(b,r))]$$

//a,b son enteros no negativos y $a \geq b$

mcd(a,b){

 i=a

 j=b

 while $j \neq 0$ do

 computar r tal que $i=qj+r$, $0 \leq r < j$

 i=j

 j=r

 end while

 return i;

}

Q: $\text{mcd}(i,j) = \text{mcd}(a,b)$

Probar que se verifica la invariante de ciclo $Q: \text{mcd}(i,j)=\text{mcd}(a,b)$

Caso base

$$Q(0): \text{mcd}(i_0, j_0) = \text{mcd}(a, b)$$

Hipótesis inductiva

$$Q(k): \text{mcd}(i_k, j_k) = \text{mcd}(a, b)$$

Paso inductivo

$$\text{Probar } Q(k+1): \text{mcd}(i_{k+1}, j_{k+1}) = \text{mcd}(a, b)$$

Por programa r_{k+1} es tal que $i_k = q_k j_k + r_{k+1}$; $i_{k+1} = j_k$; $j_{k+1} = r_{k+1}$

$$\begin{aligned} \text{mcd}(i_{k+1}, j_{k+1}) &= \text{mcd}(j_k, r_{k+1}) \quad (\text{por programa}) \\ &= \text{mcd}(i_k, j_k) \quad (\text{por programa y Euclides}) \\ &= \text{mcd}(a, b) \quad (\text{por H. I.}) \end{aligned}$$

//a,b son enteros no negativos y $a \geq b$

mcd(a,b){

i=a

j=b

while $j \neq 0$ do

computar r tal que

$i = qj + r, 0 \leq r < j$

i=j

j=r

end while

return i;

}

Probamos que la invariante es

$Q: \text{mcd}(i,j) = \text{mcd}(a,b)$

Al finalizar el ciclo while se verifica
 $j=0$.

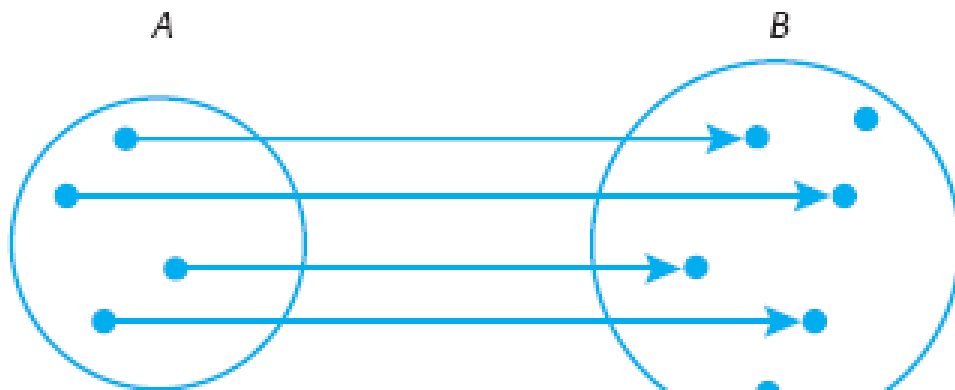
Luego $\text{mcd}(i,j) = \text{mcd}(a,b)$.
Sabemos además que $\text{mcd}(i,0)=i$.

Por lo tanto la función computa
 $\text{mcd}(a,b)$

Sea $f: A \rightarrow B$ una función. Diremos que f es **inyectiva** si y sólo si

$$\forall x, y \in A \quad x \neq y \rightarrow f(x) \neq f(y).$$

Es decir, a elementos distintos del conjunto de partida le corresponden elementos distintos del codominio,



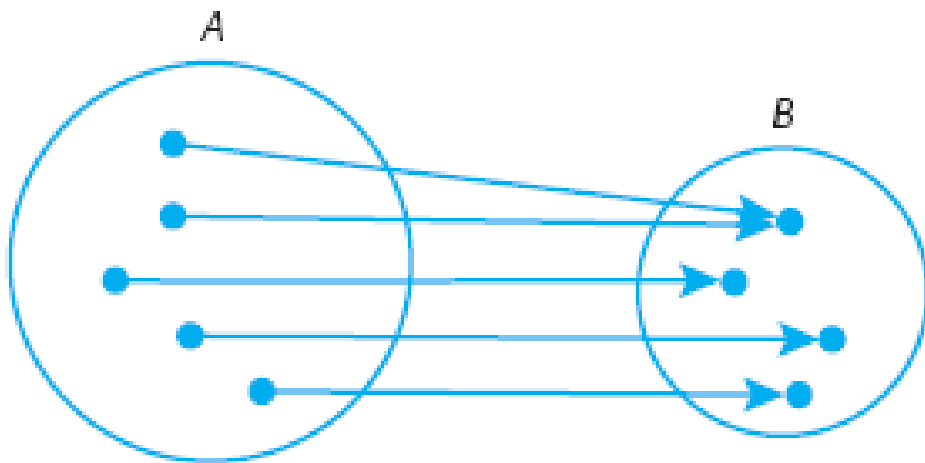
o lo que
es lo mismo

$$\forall x, y \in A \quad f(x) = f(y) \rightarrow x = y$$

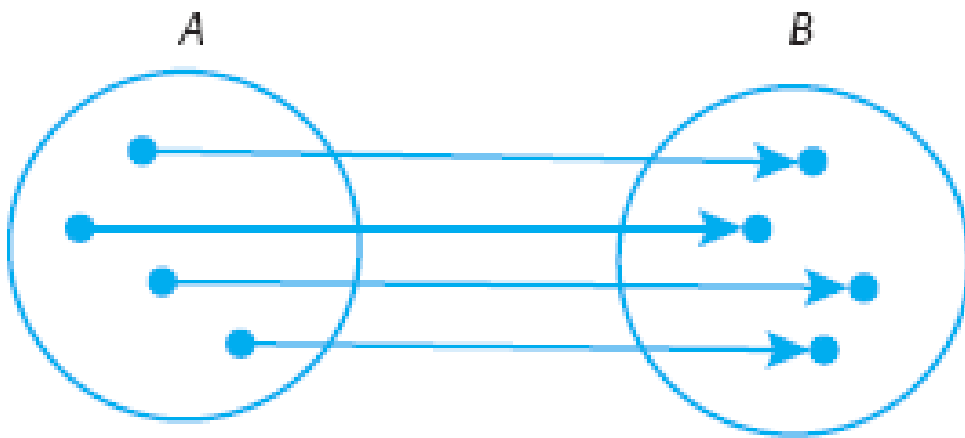
Sea $f: A \rightarrow B$ una función. Diremos que f es **sobreyectiva** si y sólo si

$$\forall y \in B \exists x \in A : f(x) = y.$$

Es decir, la imagen de f es igual al codominio de la función.



Una función es **biyectiva** si es inyectiva y sobreyectiva a la vez.



Si f es biyectiva, entonces su inversa f^{-1} es también una función y además biyectiva.

Ejemplo:

La función $f : \mathbb{R} \rightarrow \mathbb{Z}$ definida como

$$f(x) = \lceil x + 1 \rceil$$

es sobreyectiva porque para cualquier

$y \in \mathbb{Z}$ existe un número en $\mathbb{R}$, $y - 1$ tal que $f(y - 1) = y$.

Pero f no es inyectiva porque, por ejemplo, $f(2.5) = f(2.6)$.

Ejemplo:

La función $f : \mathbb{Z} \rightarrow \mathbb{Z}$ definida como

$$f(x) = x+1$$

es sobreyectiva porque para cualquier

$y \in \mathbb{Z}$ existe un número en $\mathbb{Z}$, $y - 1$ tal que $f(y - 1) = y$.

Además f es inyectiva porque, $\forall x, y \in \mathbb{Z} \quad x \neq y \rightarrow f(x) \neq f(y)$.

Concluimos que f es biyectiva.

La inversa de f , $f^{-1}: \mathbb{Z} \rightarrow \mathbb{Z}$ se define como $f^{-1}(y) = y-1$.

Ejercicios

Cuáles de las siguientes funciones son inyectivas, sobreyectivas o biyectivas (si es biyectiva encontrar la inversa):

1. $f: \mathbb{N}_3 \rightarrow \mathbb{N}_3$ definida como $f(x) = 2x \bmod 3$.

2. Sea $\Sigma = \{a, b\}$

$f: \Sigma^* \rightarrow \mathbb{N}$ definida como $f(w) = |w|$.

$f: \Sigma^* \rightarrow \mathbb{Z}$ definida como $f(w) = |w|$.

$f: \Sigma^* \rightarrow \Sigma^*$ definida como $f(w) = aw$.

Sean A y B conjuntos. Si existe una biyección entre A y B denotaremos este hecho escribiendo

$$|A| = |B|.$$

En este caso diremos que A y B tienen el *mismo tamaño* o la *misma cardinalidad* o que son *equipotentes*.

Cardinalidad de un conjunto finito

¿Cuál es la cardinalidad del conjunto A definido como sigue?

$$A = \{(x + 1)^3 \mid 1 \leq (x + 1)^3 \leq 3000, x \in \mathbb{N}\}$$

Existe una biyección $f: \{0, 1, \dots, 13\} \rightarrow A$
donde $f(x) = (x + 1)^3$.

Por lo tanto

$$|A| = |\{0, 1, \dots, 13\}| = 14.$$

Cardinalidad de un conjunto infinito

Sea Impar el conjunto de números naturales impares.

Sea f la función $f : \mathbb{N} \rightarrow \text{Impar}$ definida como $f(x) = 2x + 1$.

Dado que f es una biyección, podemos concluir que el conjunto Impar y $\mathbb{N}$ tienen el mismo tamaño. Es decir,
 $|\text{Impar}| = |\mathbb{N}|$.

Un conjunto se dice **contable** si es finito o si existe una biyección entre el mismo y $\mathbb{N}$. En el último caso se dice que el conjunto es infinitamente contable.

En términos de tamaño decimos que un conjunto S es contable si $|S| = |\{0, 1, \dots, n - 1\}|$ para algún número natural n o $|S| = |\mathbb{N}|$.

Si un conjunto no es contable, decimos que es **incontable**.

Propiedades de conjuntos contables

- a. Todo subconjunto de $\mathbb{N}$ es contable.
- b. S es contable si y sólo si $|S| \leq |\mathbb{N}|$.
- c. Cualquier subconjunto de un conjunto contable es contable.
- d. Cualquier imagen de un conjunto contable es contable.

Teorema (detalles en Hein sección 2.4.2)

$\mathbb{N} \times \mathbb{N}$ es un conjunto contable.

Prueba:

Se describe una biyección como sigue

$(0, 0), \leftrightarrow 0,$

$(0, 1), (1, 0), \leftrightarrow 1, 2$

$(0, 2), (1, 1), (2, 0), \leftrightarrow 3, 4, 5,$

$(0, n), \dots \leftrightarrow (n^2 + n)/2, \dots$

clave: $f(x,y) =$
 $((x + y)^2 + 3x + y)/2$

Teorema (detalles en Hein sección 2.4.2)

Si $S_0, S_1, \dots, S_n, \dots$ es una secuencia de conjuntos contables. Entonces la unión

$$S_0 \cup S_1 \cup \dots \cup S_n \cup \dots$$

es un conjunto contable.

clave: asociar cada tupla (m, n) en $\mathbb{N} \times \mathbb{N}$ con un elemento x_{mn} en la unión de los conjuntos dados.

Ejemplos de conjuntos contables:

Los números racionales

El conjunto Σ^* de cadenas sobre un alfabeto finito Σ .

$$\begin{array}{c|cccccc}
 & 0 & 1 & 2 & \cdots & n & \cdots \\
\hline
S_0 & a_{00} & a_{01} & a_{02} & \cdots & a_{0n} & \cdots \\
S_1 & a_{10} & a_{11} & a_{12} & \cdots & a_{1n} & \cdots \\
S_2 & a_{20} & a_{21} & a_{22} & \cdots & a_{2n} & \cdots \\
\vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\
S_n & a_{n0} & a_{n1} & a_{n2} & \cdots & a_{nn} & \cdots \\
\vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots
\end{array}$$

Podemos construir S a partir de la lista de elementos diagonales $(a_{00}, a_{11}, \dots, a_{nn}, \dots)$ cambiando cada elemento de tal manera que $a_n \neq a_{nn}$ para cada n . Entonces S difiere de cada S_n en el elemento n -ésimo. Por ejemplo, tomando dos elementos $x, y \in A$ definimos

$$a_n = \begin{cases} x & \text{si } a_{nn} = y \\ y & \text{si } a_{nn} \neq y \end{cases}$$

Podemos construir S a partir de la lista de elementos diagonales $(a_{00}, a_{11}, \dots, a_{nn}, \dots)$ cambiando cada elemento de tal manera que $a_n \neq a_{nn}$ para cada n .

	0	1	2	3	4
s_0	a_{00}				
s_1		a_{11}			
s_2			a_{22}		
s_3				a_{33}	
...					...

$$a_n = \begin{cases} x & \text{si } a_{nn} = y \\ y & \text{si } a_{nn} \neq y \end{cases}$$

	0	1	2	3	4
S_0	$a_0 \neq a_{00}$				
S_1		a_{11}			
S_2			a_{22}		
S_3				a_{33}	
...					...

$$S = \cdot a_0$$

	0	1	2	3	4
S_0	a_{00}				
S_1		$a_1 \neq a_{11}$			
S_2			a_{22}		
S_3				a_{33}	
...					...

$S = \begin{pmatrix} a_{00} & a_{01} & a_{02} & a_{03} & a_{04} \\ a_{10} & a_{11} & a_{12} & a_{13} & a_{14} \\ a_{20} & a_{21} & a_{22} & a_{23} & a_{24} \\ a_{30} & a_{31} & a_{32} & a_{33} & a_{34} \\ a_{40} & a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix}$

	0	1	2	3	4
S_0	a_{00}				
S_1		a_{11}			
S_2			a_{22} $a_2 \neq$		
S_3				a_{33}	
...					...
$S = \begin{pmatrix} \cdot & a_0 & a_1 & a_2 & & \end{pmatrix}$					

	0	1	2	3	4
S_0	a_{00}				
S_1		a_{11}			
S_2			a_{22}		
S_3				$a_3 \neq a_{33}$	
...					...
$S =$	a_0	a_1	a_2	a_3	...

Ejercicio

Sea $(0, 1) = \{x \in \mathbb{R} \mid 0 < x < 1\}$ y sea $\mathbb{R}^+$ el conjunto de reales positivos.

Mostrar que la función $f : (0, 1) \rightarrow \mathbb{R}^+$ definida como

$$f(x) = \frac{x}{1-x}$$

es una biyección.

Lema (detalles en Hein sección 2.4.3)

Los números reales son incontables

Prueba

Es suficiente probar que $U = (0,1)$ es incontable.

Asumamos por el absurdo que U es contable. Entonces podemos listar todos los números entre 0 y 1 como una secuencia contable $r_0, r_1, r_2, \dots, r_n, \dots$

Cada número real entre 0 y 1 puede ser representado como un infinito decimal. Entonces para cada n existe una representación $r_n = 0.d_{n0}d_{n1} \dots d_{nn} \dots$, donde cada d_{ni} es un dígito decimal.

Dado que r_n puede representarse por la secuencia $(0.d_{n0}d_{n1} \dots d_{nn} \dots)$, entonces, por diagonalización podemos concluir que existe un decimal infinito que no está en la lista (ver Hein).

Concluimos que U es incontable y por lo tanto $\mathbb{R}$ es incontable.

Lema (detalles en Hein sección 2.4.3)

El conjunto F de funciones de $\mathbb{N}$ en $\mathbb{N}$ es incontable

Prueba

Asumamos por el absurdo que F es contable. Entonces podemos listar todas las funciones de $\mathbb{N}$ en $\mathbb{N}$ como

$f_0, f_1, f_2, \dots, f_n, \dots$

Cada función f_n puede ser representada por la secuencia de sus valores $(f_n(0)f_n(1) \dots f_n(n) \dots)$, entonces, por diagonalización, podemos concluir que existe una función que no está en la lista (ver construcción en Hein).

Concluimos que F es incontable.

Teorema (detalles en Hein sección 2.4.4)

El conjunto de programas que pueden ser escritos utilizando un lenguajes de programación es infinitamente contable.

Prueba:

Cada programa es una cadena finita de símbolos sobre un alfabeto finito fijo. Sea P_n el conjunto de todos los programas que son cadenas de longitud n sobre A . El conjunto de todos los programas es la unión de los conjuntos $P_0, P_1, \dots, P_n, \dots$. Dado que cada P_n es finito y por lo tanto contable, podemos concluir que la unión es contable.

Existe “sólo” un número contable de programas de computadora. Por lo tanto existen límites sobre lo que puede ser computado.

Ejemplos

Dado que existe un número incontable de funciones de $\mathbb{N}$ en $\mathbb{N}$ no todas son computables. Existen programa para calcular sólo un conjunto contable de estas funciones.

Ejemplos (cont.)

No es posible computar cualquier real hasta un número arbitrario de cifras decimales. La razón es que existe un número contable de programas y un número incontable de reales.

Si extraemos los reales computables de $\mathbb{R}$, el conjunto restante sigue siendo incontable. ¿Por qué?

Los números racionales pueden ser computados.

Algunos números irracionales pueden ser computados (ejemplo: π).

Lema:

Sea $n > 1$ y sea $f : \mathbb{N}_n \rightarrow \mathbb{N}_n$ definida como sigue, donde a y b son enteros:

$$f(x) = (ax + b) \bmod n.$$

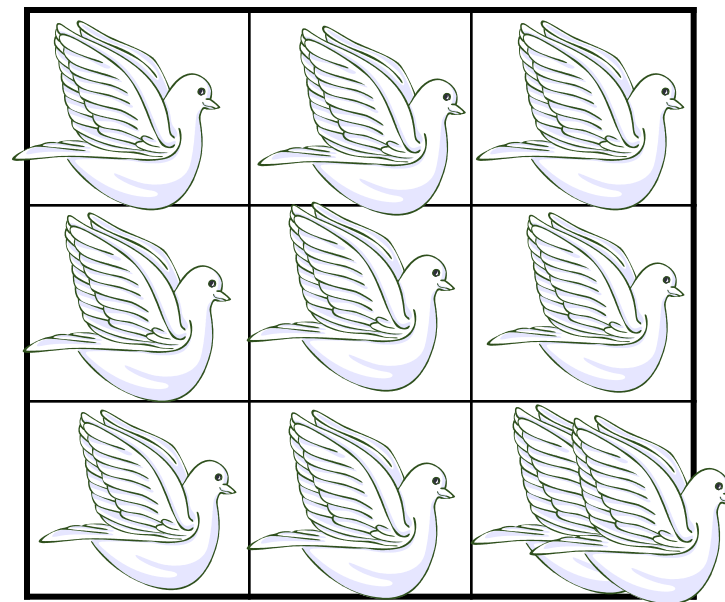
Entonces f es una biyección si y sólo si el $\text{mcd}(a,n)=1$.

Cuando esto ocurre, la función inversa f^{-1} se define como

$$f^{-1}(x) = (kx + c) \bmod n$$

donde c es un entero tal que $f(c)=0$ y k es un entero tal que $1 = ak + nm$ para algún entero m .

Si m palomas vuelan a un palomar que contiene n celdas donde $m > n$ entonces una celda tendrá dos o más palomas



Si A y B son conjuntos finitos con $|A| > |B|$, entonces toda función de A a B mapea al menos dos elementos de A a un único elemento de B . Esto es lo mismo que decir que no se puede definir una función inyectiva de A a B .

Ejemplos:

El juego de la silla, cuando juegan m personas y quedan $m-1$ sillas.

En un grupo de 8 personas dos deben haber nacido el mismo día de la semana.

En un conjunto de $n+1$ enteros existen (al menos) dos que tendrán el mismo resto al ser divididos por n .

Función de cifrado para traducir letras de un alfabeto, donde cada letra se codifica con un número del 0 al 26:

$f : \mathbb{N}_{27} \rightarrow \mathbb{N}_{27}$ definida $f(x) = (x+5) \bmod 27$

La función f “corre” la letra 5 posiciones.

Ejemplo:

Codificamos la palabra hola como mtpf

Preguntas:

¿Es f una biyección?

¿Cómo decodificamos el mensaje?

$$f^{-1}(x) = (x - 5) \bmod 27.$$

Sabemos $f(26)=4$

$$f^{-1}(4) = (4 - 5) \bmod 27 = (-1) \bmod 27 = 26$$

Criptoanálisis (del griego *kryptós*, "escondido" y *analýein*, "desatar") es el estudio de los métodos para obtener el sentido de una información cifrada, sin acceso a la información secreta requerida para obtener este sentido normalmente.

fuelle: wikipedia



Fialka (M-125)

La función $f : \mathbb{N}_{27} \rightarrow \mathbb{N}_{27}$ definida como $f(x) = (x+5) \bmod 27$ es un ejemplo de función de cifrado aditiva.

Las funciones de cifrado aditivas son monoalfabética: un carácter del alfabeto es siempre reemplazado por un mismo carácter del alfabeto.

Dado mtpf podemos deducir fácilmente que codifica la palabra hola.

Otra funciones de cifrado monoalfabéticas son las funciones de cifrado multiplicativas.

Ejemplo: $g(x) = 3x \bmod 27$

¿Es g una biyección?

Otro ejemplo de funciones de cifrado monoalfabéticas son las funciones de cifrado afines.

Utilizan un par de claves: (M, A)

Ejemplo:

clave $(4, 5)$

$f(x) = (4x + 5) \bmod 27.$

Pregunta:

¿Cómo desciframos los mensajes?

Lema

Sea $n > 1$ y sea $f : \mathbb{N}_n \rightarrow \mathbb{N}_n$ definida como sigue donde a y b son enteros:

$$f(x) = (ax + b) \bmod n.$$

Entonces f no tiene puntos fijos si y sólo si $\text{mcd}(a - 1, n)$ no divide a b .

Ejemplo

La función de cifrado $f : \mathbb{N}_{27} \rightarrow \mathbb{N}_{27}$

$$f(x) = (4x + 5) \bmod 27$$

no tiene puntos fijos porque:

$\text{mcd}(4-1, 27) = \text{mcd}(3, 27) = 3$, y 3 no divide a 5.

Por otra parte, la función de cifrado

$$f(x) = (4x + 6) \bmod 27$$

tiene puntos fijos. ¿Cuáles son?

Una **función hash** es una función que mapea un conjunto S de claves a un conjunto finito de índices $0, 1, \dots, n - 1$ de una tabla.

Una tabla cuya información se encuentra utilizando una función de hash se denomina **tabla hash**.

Ejemplo:

Mapear las abreviaturas de los meses del año utilizando la función de hash $f: S \rightarrow \{0,1,\dots,11\}$ de la siguiente manera.

$f(XYZ) = (\text{ord}(X) + \text{ord}(Y) + \text{ord}(Z)) \bmod 12$
donde $\text{ord}(X)$ el código ASCII de X .

Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
5	5	0	3	7	1	11	9	8	6	7	4

Colisiones:

Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
5	5	0	3	7	1	11	9	8	6	7	4

¿Cómo asignar un nuevo valor?

Sondeo Lineal:

Buscar hasta encontrar un espacio libre:

$(k + 1) \bmod n, (k + 2) \bmod n, \dots, (k + n) \bmod n.$

Preguntas:

- ¿Es posible encontrar una función inyectiva de tal manera que no existan colisiones?
- ¿Si incrementamos el tamaño de la tabla, sería más simple encontrar una función inyectiva?
- ¿Si el tamaño de la tabla es incrementado, podemos dispersar los elementos de tal manera que las colisiones puedan ser resueltas en menor tiempo?

Sondeo lineal utilizando huecos (gaps)

Buscar hasta encontrar un espacio libre:

$(k + g) \bmod n, (k + 2g) \bmod n, \dots, (k + ng) \bmod n.$

Ejemplos:

Para $n = 12$ y $g = 4$

$k = 7$:

11, 3, 7, 11, 3, 7, 11, 3, 7, 11, 3, 7.

Para $n = 12$ y $g = 5$

$k=7$

0, 5, 10, 3, 8, 1, 6, 11, 4, 9, 2, 7.

Si queremos
encontrar una
biyección
necesitamos
 $\text{mcd}(g,n)=1$
 $(1 \leq g < n)$

Aplicaciones de las funciones hash:

- Acceso rápido a archivos.
- Identificar archivos (por ejemplo, usado en sistemas P2P).
- Corroborar que un archivo no haya cambiado (test de integridad).

Sea suc la función sucesor
 suc es tal que $\text{suc}(x) = x+1$.

Definición recursiva de suma

Podemos definir la función suma recursivamente como sigue:

$$\text{suma}(m, 0) = m$$

$$\text{suma}(m, n+1) = \text{suc}(\text{suma}(m, n))$$

Definición recursiva de producto:

$$\text{prod}(m, 0) = 0$$

$$\text{prod}(m, n+1) = \text{suma}(m, \text{prod}(m, n))$$

Definición recursiva de exponenciación:

$$\text{exp}(m, 0) = 1$$

$$\text{exp}(m, n+1) = \text{prod}(m, \text{exp}(m, n))$$

Nota: usaremos $+$, $*$ y $\uparrow$ para denotar suma, producto y exponenciación, respectivamente.

Suma de números impares

$$\begin{aligned}
 f(n) &= 1 + 3 + \dots + (2n + 1) \\
 &= (1 + 3 + \dots + (2n - 1)) + (2n + 1) \\
 &= (1 + 3 + \dots + 2(n - 1) + 1) + (2n + 1) \\
 &= f(n - 1) + 2n + 1.
 \end{aligned}$$

Definición recursiva de f

$$\begin{aligned}
 f(0) &= 1, \\
 f(n + 1) &= f(n) + 2n + 3.
 \end{aligned}$$

Definición recursiva de fibonacci

$$\text{fib}(0) = 0,$$

$$\text{fib}(1) = 1,$$

$$\text{fib}(n+2) = \text{fib}(n+1) + \text{fib}(n)$$

Al trabajar con cadenas, asumiremos que nuestro alfabeto está compuesto sólo por los símbolos a y b .

Sea complemento una función definida sobre cadenas tal que transforma las a 's en b 's y viceversa.

Definición recursiva de complemento

$$\text{comp}(\lambda) = \lambda$$

$$\text{comp}(a.w) = b.\text{comp}(w)$$

$$\text{comp}(b.w) = a.\text{comp}(w)$$

Notación

$L = []$ representa la lista vacía

$L = x :: T$ representa la lista L con cabeza (primer elemento) x y cola T .

Definición recursiva de longitud

$$\text{longitud} ([]) = 0$$

$$\text{longitud} (a::T) = 1 + \text{longitud} (T) .$$

$\text{pares}([a, b, c], [1, 2, 3]) = [(a, 1), (b, 2), (c, 3)]$

Definición recursiva de pares

$\text{pares}([], []) = []$,

$\text{pares}(a :: T, b :: T') = (a, b) :: \text{pares}(T, T')$.